

BAHASA PEMROGRAMAN INDONESIA

Reaktivitas Jantung KARSA

Modul keempat dari seri pembelajaran KARSA v0.3.1. Pelajari model reaktivitas signal-based, computed properties, watcher, dan visualisasi dependency graph yang menjadi inti dari setiap aplikasi KARSA reaktif.

● Level Menengah

● 30 Jam Belajar

● 9 Bab + Mini-Proyek

● 15+ Latihan

Daftar Isi

Modul 4 — Reaktivitas: Jantung KARSA · v0.3.1

PENGANTAR

Pengantar Modul & Mind Map	03
Tujuan, prasyarat, peta 9 bab	

BAB UTAMA

Bab 1 — Filosofi Reaktivitas KARSA	05
Signal-based, Proxy + .value, beda vs Vue/React	
Bab 2 — data — State Reaktif	08
Deklarasi, deep reactive, beda data/tetap/ubah	
Bab 3 — turunan — Computed Properties	11
Auto-dependency, lazy eval, read-only	
Bab 4 — saat — Watcher	15
Side effect, batched flush, anti-pattern	
Bab 5 — Mutasi State: 4 Mutator	18
simpan, tambahkan, kurangi, sisipkan	
Bab 6 — Reaktivitas Array & Object	21
Tracking per-key, spread trigger, deep reactive	
Bab 7 — Dependency Graph — Visualisasi	24
karsa graph, baca graph, deteksi cycle	
Bab 8 — Inspect Mode — Debug Reaktivitas	27
karsa inspect, SemanticSymbol, readCount/writeCount	
Bab 9 — Mini-Proyek: Counter Multi-State	30
Step-by-step dari example/index.ks	

LATIHAN & SOLUSI

Latihan Soal (15+ soal bertingkat)	35
5 mudah, 5 sedang, 5 sulit	
Solusi Latihan Lengkap	41
Kode + penjelasan + alternatif + pembahasan error	

Lampiran A — Referensi Cepat

Cheat sheet semua keyword Modul 4

46

Lampiran B — Perbandingan Framework Reaktif

KARSA vs Vue vs React vs Solid

47

Lampiran C — Schema SemanticSymbol & Dependency

Field lengkap dari karsa inspect & graph

48

Lampiran D — Error Codes Reaktivitas

E3003, E4002, E4004, E4201, W3003, W41xx

49

Selamat Datang di Modul 4

Modul 4 membawa Anda ke inti dari KARSA: reaktivitas. Setelah mempelajari DOM & Event di Modul 3, kini saatnya memahami bagaimana UI bisa "bereaksi" otomatis ketika state berubah. Inilah yang membedakan aplikasi modern dari manipulasi DOM manual. KARSA v0.3.1 mengadopsi model signal-based yang deterministic dan batched, lebih sederhana dari React hooks, lebih transparan dari Vue reactivity.

Reaktivitas KARSA dibangun di atas empat pilar: `data` untuk state reaktif, `turunan` untuk computed properties, `saat` untuk watcher, dan empat mutator (`simpan`, `tambahkan`, `kurangi`, `sisipkan`) untuk perubahan state. Di balik layar, Proxy JavaScript melacak setiap akses properti untuk membangun dependency graph otomatis. Tidak ada boilerplate `useEffect` dengan dependency array manual seperti React, tidak ada `watch` eksplisit seperti Vue.

1 Prasyarat

Sebelum memulai, pastikan Anda telah menyelesaikan **Modul 1: Dasar KARSA**, **Modul 2: Logika & Kontrol Alur**, dan **Modul 3: DOM & Event**. Anda harus memahami: (1) deklarasi `data` / `tetap` / `ubah` dan type hint; (2) konsep scope lokal vs global; (3) manipulasi DOM dengan `buat`, `perbarui`, dan `ketika`; (4) lifecycle komponen. Tanpa pondasi ini, konsep reaktivitas akan terasa abstrak.

2 Kompetensi yang Diraih

Setelah menyelesaikan Modul 4, Anda diharapkan mampu: (1) menjelaskan model reaktivitas signal-based KARSA dan bedanya dengan Vue/React; (2) mendeklarasikan state reaktif dengan `data`; (3) membuat computed properties dengan `turunan` dan memahami auto-dependency tracking; (4) memasang side effect dengan `saat` dan menghindari infinite loop; (5) memutasikan state dengan empat mutator dan memahami perilaku per tipe; (6) mengelola reaktivitas array dan object termasuk pola spread trigger; (7) membaca dan memvisualisasikan dependency graph via `karsa graph`; (8) debug reaktivitas dengan `karsa inspect`; (9) mendeteksi dan memecahkan dependency cycle.



DURASI
30 Jam

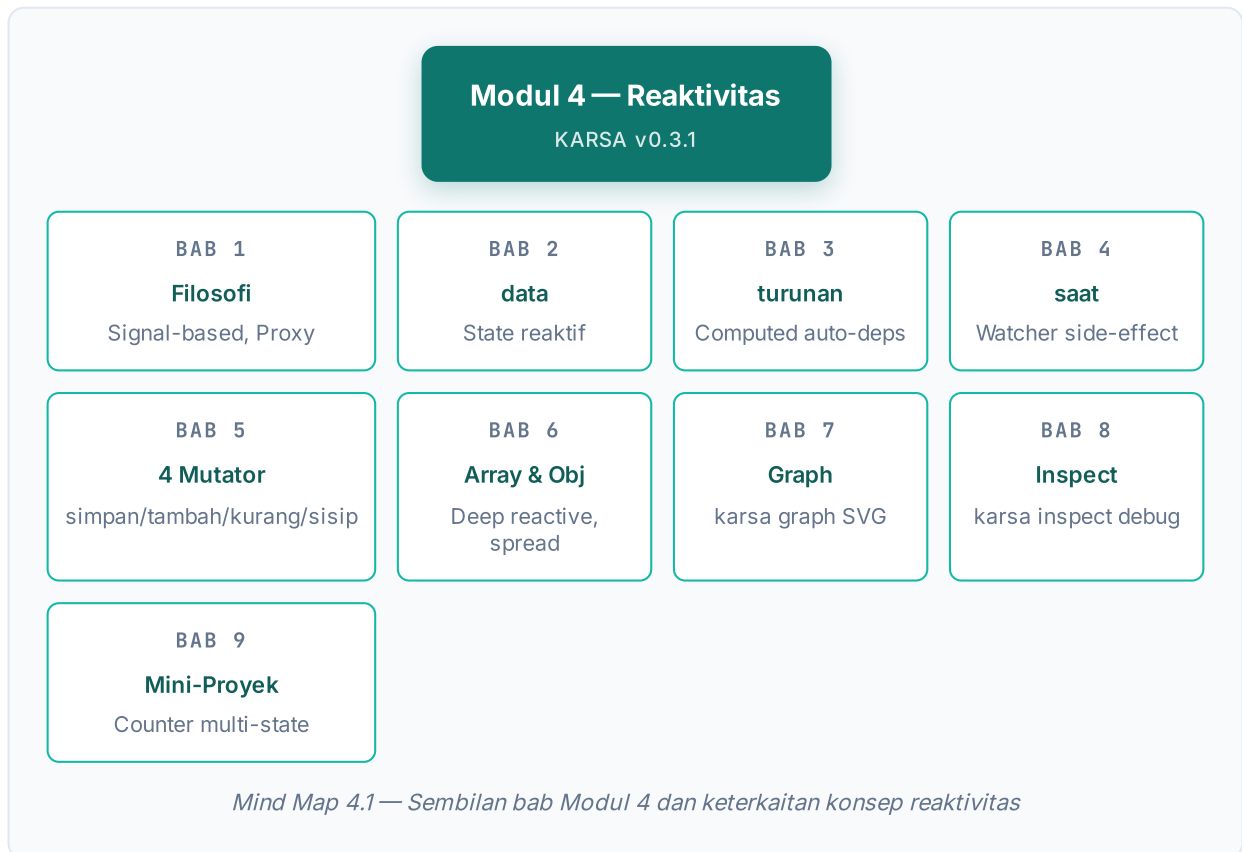


BAB
9 + Proyek



LATIHAN
15+ Soal

3 Peta Konsep Modul 4



Peta di atas menggambarkan alur belajar. Bab 1 meletakkan fondasi filosofis. Bab 2–4 memperkenalkan tiga kata kunci inti: `data`, `turunan`, `saat`. Bab 5–6 membahas mutasi state dan reaktivitas struktur data kompleks. Bab 7–8 memperkenalkan tooling profesional untuk visualisasi dan debugging. Bab 9 mengintegrasikan semuanya dalam mini-proyek Counter Multi-State.

BAB 1

Filosofi Reaktivitas KARSA

Reaktivitas adalah paradigma di mana UI otomatis diperbarui ketika state berubah, tanpa developer harus manual memanipulasi DOM. KARSA v0.3.1 mengadopsi model **signal-based reactivity** yang deterministic dan batched, berbeda dari React (hooks + virtual DOM diff) maupun Vue (Proxy-based dengan getter/setter tracking). Memahami filosofi ini penting agar Anda bisa mendiagnosis masalah reaktivitas dengan tepat.

1.1 Apa itu Reaktivitas?

Dalam pemrograman imperatif tradisional, ketika sebuah nilai berubah, Anda harus manual memanggil fungsi untuk memperbarui tampilan. Misalnya, jika `counter` berubah dari 5 ke 6, Anda harus memanggil `document.querySelector("#angka").innerText =`

`counter` agar UI ikut berubah. Ini error-prone: lupa satu pembaruan, UI jadi tidak sinkron dengan state.

Reaktivitas membalik paradigma ini. Anda deklarasikan *"UI ini bergantung pada state X"*, lalu sistem otomatis memperbarui UI ketika X berubah. Anda tidak perlu tahu *kapan* harus update — sistem yang melacaknya. Hasilnya: kode lebih deklaratif, lebih sedikit bug sinkronisasi, lebih mudah dirawat.

1.2 Model Signal-Based KARSA

KARSA menggunakan **signal-based reactivity** yang diimplementasikan dengan `Proxy` JavaScript. Setiap `data` di-wrap menjadi objek dengan properti `.value`, dan akses ke `.value` otomatis terlacak sebagai dependency.

```
JAVASCRIPT · RUNTIME INTERNAL __CREATEREACTIVE

function __createReactive(val) {
  const obj = { value: val, __id: ++__effectId };
  const proxy = new Proxy(obj, {
    get(target, prop) {
      // Saat .value dibaca, cat sebagai dependency
      if (__activeEffect && prop === 'value') {
        let subs = __subscribers.get(proxy) || new Set();
        subs.add(__activeEffect);
        __subscribers.set(proxy, subs);
      }
      return target[prop];
    },
    set(target, prop, newVal) {
      const oldVal = target[prop];
      target[prop] = newVal;
      // Saat .value ditulis, jalankan semua subscriber
      if (prop === 'value' && oldVal !== newVal) {
        const subs = __subscribers.get(proxy);
        if (subs) {
          subs.forEach(fn => fn(newVal, oldVal));
        }
      }
      return true;
    }
  });
  return proxy;
}
```

Mekanisme sederhana: setiap kali `.value` dibaca dalam konteks efek aktif (computed atau watcher), efek tersebut dicatat sebagai subscriber. Ketika `.value` ditulis, semua subscriber dijalankan. Inilah inti reaktivitas KARSA.

1.3 Batched Microtask Flush

KARSA tidak menjalankan subscriber secara sinkron saat setiap write. Sebaliknya, perubahan di-batch dalam satu *microtask flush*. Artinya, jika Anda mutate `counter` tiga kali berturut-turut dalam satu event handler, watcher hanya dipicu sekali dengan nilai akhir, bukan tiga kali. Ini penting untuk performa dan prediktabilitas.

● KENAPA BATCHED?

Bayangkan Anda punya 5 state berubah dalam satu handler. Tanpa batching, setiap perubahan memicu re-render UI — total 5 render. Dengan batching, semua perubahan dikumpulkan, lalu re-render hanya sekali dengan state final. Hasil: UI terasa lebih responsif, tidak ada flicker, dan logika tetap predictable.

1.4 Beda KARSA vs Vue vs React

ASPEK	KARSA V0.3.1	VUE 3	REACT
State reaktif	<code>data x = 0</code>	<code>const x = ref(0)</code>	<code>const [x, setX] = useState(0)</code>
Akses nilai	<code>x</code> (auto-unwrap)	<code>x.value</code>	<code>x</code> (langsung)
Mutasi	simpan 5 ke <code>x</code>	<code>x.value = 5</code>	<code>setX(5)</code>
Computed	<code>turunan y = x * 2</code>	<code>const y = computed(() => x.value * 2)</code>	<code>const y = useMemo(() => x * 2, [x])</code>
Watcher	saat <code>x</code> berubah: <code>...</code>	<code>watch(x, (...))</code>	<code>useEffect(() => ..., [x])</code>
Dep tracking	Otomatis via Proxy	Otomatis via Proxy	Manual dependency array
Batching	Microtask flush	nextTick + scheduler	Concurrent rendering
Boilerplate	Minimal	Sedang	Banyak (hooks, deps array)

● FILOSOFI KARSA

KARSA sengaja memilih model signal-based dengan auto-unwrap karena: (1) **keterbacaan** — kode terbaca alami tanpa `.value` berulang; (2) **auto-dependency tracking** — tidak ada dependency array manual yang rentan bug; (3) **predictable batching** — microtask flush lebih sederhana dipahami daripada concurrent rendering React.

1.5 Diagram Alur Reaktivitas

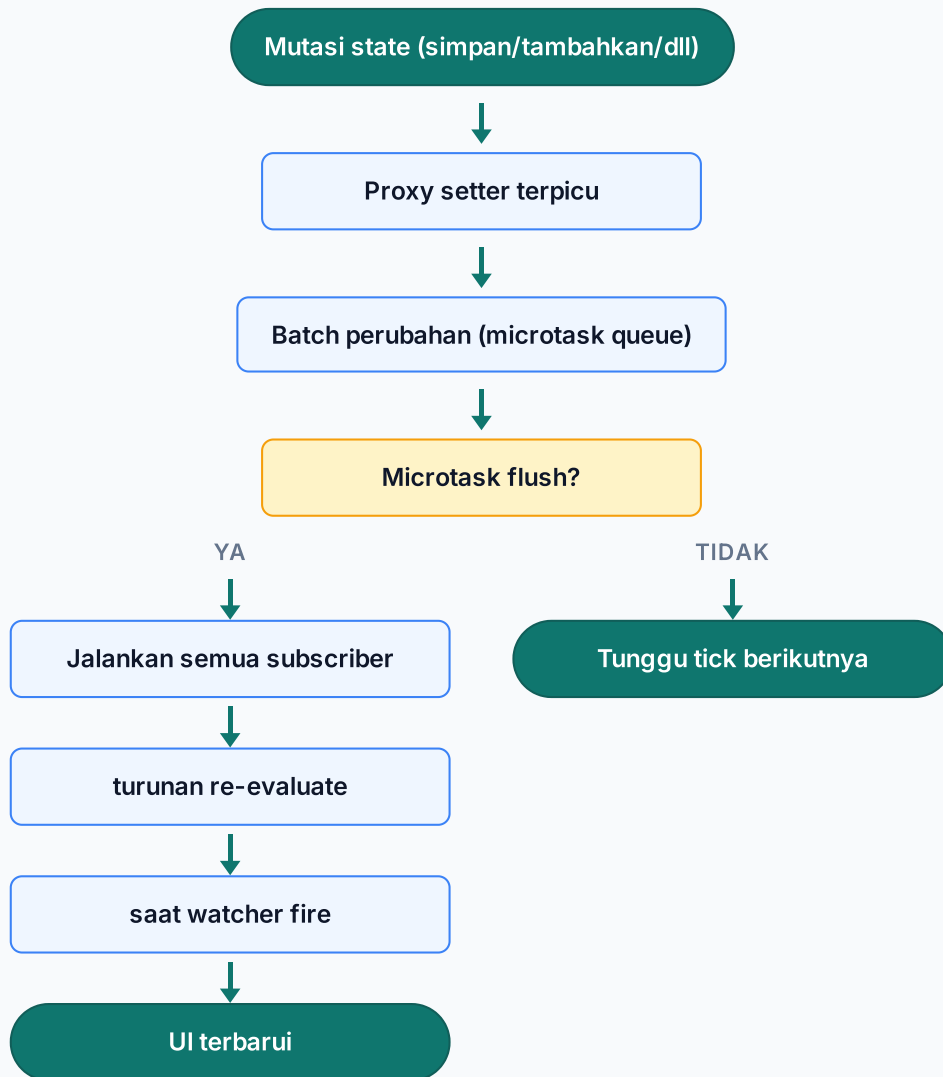


Diagram 4.1 — Alur reaktivitas KARSA dari mutasi state hingga UI terbaru

TUJUAN BELAJAR

Peserta memahami konsep reaktivitas, model signal-based KARSA, dan bisa membandingkan dengan Vue/React.

SARAN MENGAJAR

Demo live: kode imperative (manual update DOM) vs reaktif KARSA. Tunjukkan baris kode yang berkurang. Lalu demo batching: mutate state 3x, lihat watcher fire 1x via console.log.

KESALAHAN UMUM

- Mengira reaktivitas = virtual DOM (beda konsep)
- Bingung kenapa **turunan** tidak re-evaluate saat dependency tidak berubah
- Mengharapkan watcher fire secara sinkron (sebenarnya batched)

TROUBLESHOOTING

- UI tidak update → cek apakah state benar **data** (bukan **tetap** / **ubah**)
- Watcher fire berulang → cek mutasi state yang sama di dalam watcher

PERTANYAAN DISKUSI

1. Apa keuntungan auto-unwrap vs .value Vue? 2. Kapan batching jadi masalah? 3. Mengapa React pakai dependency array manual?

BAB 2

data — State Reaktif

data adalah kata kunci untuk mendeklarasikan state reaktif. Setiap variabel yang dideklarasikan dengan **data** otomatis menjadi *signal* — perubahan nilainya akan memicu semua **turunan** dan **saat** yang membacanya. Inilah pondasi dari setiap aplikasi reaktif KARSA.

2.1 Sintaks Dasar

```
--! Deklarasi data reaktif
data counter = 0
data nama = "Budi"
data daftar = [1, 2, 3]
data pengguna = { nama: "Budi", skor: 0 }

--! Dengan type hint (opsional)
data umur: angka = 25
data email: teks = "budi@example.com"
data aktif: benar-salah = benar
```

Sintaks: **data** <nama> [: <tipe>] = <nilai_awal>. Type hint opsional — jika diberikan, compiler bisa memberi warning (W4001) jika nilai tidak cocok dengan tipe. Type yang didukung: **angka**, **teks**, **benar-salah**, **objek**, **array**, **elemen**, **fungsi**,

apapun .

2.2 Deep Reactive by Default

Berbeda dari React yang menggunakan shallow state, `data` KARSA bersifat **deep reactive**. Artinya, properti nested dari objek atau elemen array juga otomatis reaktif. Anda bisa mutate `pengguna.skor` atau `daftar[0]` dan perubahan akan terlacak.

```
KARSA · 02-DATA-DEEP.KS

data pengguna = {
  nama: "Budi",
  skor: 0,
  alamat: { kota: "Jakarta" }
}

--! Mutasi nested - tetap reaktif
simpan 100 ke pengguna.skor
simpan "Bandung" ke pengguna.alamat.kota

--! turunan yang baca nested property akan re-evaluate
turunan info = pengguna.nama + " - " + pengguna.alamat.kota
```

2.3 Beda data vs tetap vs ubah

KARSA menyediakan tiga kata kunci deklarasi variabel. Pilih yang tepat sesuai kebutuhan reaktivitas.

KATA KUNCI	PADANAN JS	REAKTIF?	USE CASE
<code>data</code>	<code>let</code> + Proxy	Ya (deep)	State UI yang butuh reaktivitas (counter, form, list)
<code>tetap</code>	<code>const</code>	Tidak	Konstanta, konfigurasi yang tidak berubah
<code>ubah</code>	<code>let</code>	Tidak	Variabel mutable non-reaktif (counter loop, temp)

● SALAH PILIH = UI TIDAK UPDATE

Kesalahan umum: pakai `ubah` untuk state UI karena "tidak butuh Proxy overhead". Akibatnya, UI tidak update saat nilai berubah. Aturan praktis: jika variabel akan ditampilkan di UI atau mempengaruhi tampilan, gunakan `data`. `ubah` hanya untuk variabel internal algoritma yang tidak terkait UI.

2.4 Runtime Internals — `__createReactive`

Saat compiler KARSA memproses `data counter = 0`, ia menurunkannya menjadi:

```
JAVASCRIPT · OUTPUT KOMPILASI (DARI COUNTER-FULL.SNAP.JS)

// @karsa-source 2:1 DataDeclaration
const hitungan = __createReactive(0);
```

`hitungan` adalah Proxy yang membungkus `{ value: 0, __id: ... }`. Saat kode KARSA mengakses `hitungan`, compiler meng-emit `hitungan.value` (auto-unwrap). Saat kode KARSA menulis `simpan 5 ke hitungan`, compiler meng-emit `__setState(hitungan, 5)` yang memanggil setter Proxy.

● AUTO-UNWRAP DI KARSA

Berbeda dari Vue yang mengharuskan `x.value` di JavaScript, KARSA meng-unwrap otomatis. Anda tulis `counter` di kode KARSA, compiler terjemahkan ke `counter.value` di JavaScript. Ini membuat kode KARSA lebih bersih dan alami.

◆ CATATAN PENGAJAR — BAB 2

DURASI 3 JAM · TEORI + PRAKTIK

TUJUAN BELAJAR

Peserta memahami data sebagai state reaktif, deep reactive behavior, dan bisa memilih antara data/tetap/ubah.

SARAN MENGAJAR

Demo: counter sederhana dengan data vs ubah. Tunjukkan UI tidak update untuk ubah. Lalu demo deep reactive dengan nested object.

KESALAHAN UMUM

- Pakai `ubah` untuk state UI → tidak reaktif
- Lupa bahwa `tetap` tidak bisa diubah (E3003)
- Bingung kenapa mutasi nested tidak terdeteksi (sebenarnya deep reactive)

TROUBLESHOOTING

- **E3003**: Menulis ke `tetap`
- **W4001**: Type hint tidak cocok dengan nilai
- UI tidak update → cek `data` vs `ubah`

PERTANYAAN DISKUSI

1. Kapan harus pakai tetap vs data? 2. Apa overhead Proxy untuk deep reactive? 3. Bisakah data berisi fungsi?

BAB 3

turunan — Computed Properties

`turunan` mendeklarasikan computed properties — state yang nilainya dihitung otomatis dari state lain. Tidak perlu manual update, tidak perlu dependency array. KARSA melacak dependency otomatis via Proxy saat `turunan` pertama kali dievaluasi. Inilah salah satu fitur paling powerful di KARSA.

3.1 Sintaks Dasar

```
KARSA · 03-TURUNAN-DASAR.KS

data harga = 10000
data jumlah = 3

--! Computed: dihitung ulang saat harga/jumlah berubah
turunan total = harga * jumlah
turunan diskon = total * 0.1
turunan total_akhir = total - diskon

--! Bisa pakai jika/kondisi
turunan status = jika total_akhir > 50000: "Mahal" jika tidak: "Terjangkau"
```

`turunan total = harga * jumlah` akan otomatis re-evaluate ketika `harga` atau `jumlah` berubah. Bahkan `total_akhir` yang bergantung pada `total` dan `diskon` (yang juga turunan) akan re-evaluate secara berantai. Semua ini otomatis, tanpa kode tambahan.

3.2 Auto-Dependency Tracking

Bagaimana KARSA tahu `total` bergantung pada `harga` dan `jumlah`? Saat `turunan total` pertama kali dievaluasi, KARSA menyetel `__activeEffect` ke computed effect. Setiap kali `.value` dari `harga` atau `jumlah` dibaca selama eksekusi, computed effect dicatat sebagai subscriber. Saat `harga` berubah, subscriber dijalankan — `total` dihitung ulang.

```
JAVASCRIPT · OUTPUT KOMPILASI __CREATECOMPUTED

function __createComputed(fn) {
  const reactive = __createReactive(null);
  const effect = function computedEffect() {
    __activeEffect = effect;
    try { reactive.value = fn(); } catch(e) { /* defer */ }
    __activeEffect = null;
  };
  effect.__deps = new Set();
  effect.__isComputed = true;
  effect(); // evaluasi awal, track dependency
  __effectMap.set(reactive, effect);
  return reactive;
}

// Penggunaan:
const total = __createComputed(() => harga.value * jumlah.value);
```

3.3 Aturan Penting — Read-Only & No Side-Effect

● DUA ATURAN KERAS

`turunan` punya dua aturan yang tidak bisa dilanggar:

- **E4004**: Tidak boleh menulis ke `turunan`. `turunan` bersifat read-only karena nilainya diturunkan dari state lain. `simpan 5 ke total` akan menghasilkan error.
- **E4002**: Tidak boleh ada side-effect di ekspresi `turunan`. Anda tidak bisa memanggil `tampilkan pesan`, `perbarui`, atau operasi DOM lain di dalam `turunan`. `turunan` harus pure function dari state input.

```
KARSA · 04-TURUNAN-ATURAN.KS

--! SALAH: side-effect di turunan
turunan status = tampilkan pesan "hitung" -- E4002

--! SALAH: write ke turunan
simpan 5 ke total -- E4004

--! BENAR: pure expression
turunan status = jika total > 50000: "Mahal" jika tidak: "Terjangkau"

--! BENAR: panggil fungsi native (yang pure)
fungsi format_rupiah(n: angka) → teks:
  kembalikan "Rp" + n
turunan total_format = format_rupiah(total)
```

3.4 Lazy Evaluation

`turunan` di KARSA bersifat *eager* — dievaluasi saat pertama kali dideklarasikan, lalu re-evaluate saat dependency berubah. Tapi nilai yang tidak dibaca tidak akan memicu re-evaluation berantai sampai ada yang membacanya (lazy chain). Ini berbeda dari React `useMemo` yang re-evaluate setiap render terlepas dibaca atau tidak.

```

data a = 1
data b = 2

--! turunan A bergantung pada a
turunan sum_ab = a + b

--! turunan B bergantung pada sum_ab
turunan double_sum = sum_ab * 2

--! Jika hanya `a` berubah dan `double_sum` tidak dibaca:
--! - sum_ab re-evaluate (dependency berubah)
--! - double_sum ditandai dirty tapi tidak langsung evaluate
--! - double_sum evaluate saat dibaca

simpan 10 ke a -- a=10, sum_ab=12 (re-eval), double_sum dirty
tampilkan pesan double_sum -- sekarang evaluate: 24

```

3.5 Contoh Realistis dari Repo

Berikut potongan dari `example/index.ks` — aplikasi counter reaktif dengan `turunan` untuk status genap/ganjil:

```

--! Fungsi JS untuk cek ganjil/genap
langsung:
function cekGenap(n) {
  return n % 2 === 0 ? "Genap" : "Ganjil";
}

data counter = 0
data nama_pengguna = "Kawan"

--! turunan: status berubah otomatis saat counter berubah
turunan status_counter = jalankan cekGenap dengan counter

--! turunan: pesan berubah saat nama_pengguna berubah
turunan pesan_selamat = "Halo, " + nama_pengguna + "!"

```

TURUNAN + JALANKAN

`turunan` bisa memanggil fungsi JS via `jalankan` selama fungsi tersebut pure (tidak ada side-effect). `cekGenap` menerima angka, mengembalikan string — perfect untuk computed. Hindari `jalankan` ke fungsi yang mutate state global atau berinteraksi dengan DOM.

TUJUAN BELAJAR

Peserta memahami turunan, auto-dependency tracking, aturan read-only & no side-effect, dan lazy chain.

SARAN MENGAJAR

Demo: counter dengan turunan status. Ubah counter, lihat status re-evaluate. Lalu trigger E4004 dengan coba simpan ke turunan.

KESALAHAN UMUM

- Coba **simpan** ke turunan → E4004
- Sisipkan **tampilkan** di turunan → E4002
- Bingung kenapa turunan tidak update (cek dependency terbaca)

TROUBLESHOOTING

- **E4002**: Hapus side-effect dari turunan
- **E4004**: Ganti **turunan** ke **data** jika butuh writable
- **E4201**: Cycle — lihat Bab 7

PERTANYAAN DISKUSI

1. Kapan pakai turunan vs data? 2. Apa beda lazy vs eager evaluation? 3. Bisakah turunan memanggil fungsi dengan side-effect?

BAB 4

saat — Watcher

saat adalah kata kunci untuk memasang *watcher* — side effect yang dijalankan otomatis ketika state reaktif berubah. Berbeda dari **turunan** yang menghasilkan nilai, **saat** menjalankan aksi. Inilah tempat Anda memperbarui DOM, memicu fetch, atau menjalankan logika imperatif yang merespons perubahan state.

4.1 Sintaks Dasar

```
data counter = 0

--! Watcher: jalankan saat counter berubah
saat counter berubah:
  perbarui teks "#angka" → counter
  jika counter ≥ 10:
    tampilkan pesan "Sudah mencapai 10!"
```

Sintaks: **saat** <identifikasi> **berubah:** lalu blok aksi. Identifikasi harus berupa **data** reaktif atau **turunan**. Jika bukan reaktif (misal **ubah**), analyzer mengeluarkan warning W3003 atau W4104.

4.2 Bedanya dengan turunan

ASPEK	TURUNAN	SAAT
Menghasilkan nilai?	Ya (state baca-saja)	Tidak (hanya aksi)
Side-effect?	Dilarang (E4002)	Boleh (memang tujuannya)
Dipakai di ekspresi?	Ya, sebagai nilai	Tidak, hanya efek samping
Use case	Hitung nilai turunan	Update DOM, fetch, log
Re-evaluate saat?	Dependency berubah	Target berubah

4.3 Runtime Internals — __watch

```
JAVASCRIPT · OUTPUT KOMPILESI __WATCH

function __watch(reactive, cb) {
  const effect = function watchEffect(n, o) { cb(n, o); };
  effect.__deps = new Set();
  let subs = __subscribers.get(reactive) || new Set();
  subs.add(effect);
  __subscribers.set(reactive, subs);
  return function unsubscribe() {
    const subs = __subscribers.get(reactive);
    if (subs) subs.delete(effect);
  };
}

// Penggunaan:
__watch(hitungan, (nilaiBaru, nilaiLama) => {
  document.querySelector("#angka").innerText = hitungan.value;
});
```

4.4 Batched Execution

Sama seperti `turunan`, `saat` juga batched. Jika Anda mutate `counter` tiga kali dalam satu handler:



```
data counter = 0

saat counter berubah:
  tampilkan pesan "Counter: " + counter

--! Handler yang mutate 3x
ketika #btn diklik:
  simpan 1 ke counter
  simpan 5 ke counter
  simpan 10 ke counter
--! Watcher fire SEKALI dengan nilai 10, bukan 3x
```

4.5 Anti-Pattern — Infinite Loop

● BAHAYA: MUTASI STATE YANG SAMA DI WATCHER

Jika Anda mutate state yang sama di dalam watcher-nya sendiri, Anda membuat loop tak terhingga. KARSA mendeteksi pola ini dan men-defer ke batch berikutnya, tapi loop tetap terjadi.



```
--! ANTI-PATTERN: infinite loop
data counter = 0

saat counter berubah:
  simpan counter + 1 ke counter -- !! loop forever

--! SOLUSI: pakai kondisi exit
saat counter berubah:
  jika counter < 100:
    simpan counter + 1 ke counter
```

4.6 Contoh Realistis — Sinkronisasi UI



```
--! Sinkronisasi counter ke UI
saat counter berubah:
  perbarui teks "#nilai-text" → "Nilai saat ini: " + counter
  perbarui teks "#status-text" → "Status angka: " + status_counter

--! Sinkronisasi pesan ke UI
saat pesan_selamat berubah:
  perbarui teks "#judul" → pesan_selamat
```

TUJUAN BELAJAR

Peserta memahami saat sebagai watcher, bedanya dengan turunan, batching, dan bisa menghindari infinite loop.

SARAN MENGAJAR

Demo infinite loop: pasang saat yang mutate state yang sama, lihat browser hang. Lalu fix dengan kondisi exit. Demo batching: mutate 3x, lihat fire 1x.

KESALAHAN UMUM

- Mutasi state yang sama di watcher → infinite loop
- Pakai **saat** padahal butuh **turunan** (bisa pakai nilai)
- Side-effect berat di watcher → lag

TROUBLESHOOTING

- **W3003**: Watcher target bukan data reaktif (resolver)
- **W4104**: Watcher target bukan data reaktif (analyzer)
- Loop tak terhingga → cek mutasi state yang sama

PERTANYAAN DISKUSI

1. Kapan pakai saat vs turunan? 2. Bagaimana cara batasi watcher agar tidak loop? 3. Apa beda watcher sync vs async?

BAB 5

Mutasi State: 4 Mutator

KARSA menyediakan empat kata kunci mutator untuk mengubah state reaktif: **simpan** (assignment), **tambahkan** (add/append/concat), **kurangi** (subtract), dan **sisipkan** (array insert). Setiap mutator memicu reaktivitas secara otomatis. Perilaku mutator bergantung pada tipe target — angka, array, atau string.

5.1 **simpan** — Assignment Langsung

```
--! Assignment ke data reaktif
simpan 5 ke counter
simpan "HaLo" ke nama
simpan [1, 2, 3] ke daftar

--! Assignment ke nested property
simpan 100 ke pengguna.skor
simpan "Bandung" ke pengguna.alamat.kota
```

simpan melakukan assignment langsung ke **.value** dari Proxy. KARSA meng-emit **__setState(nama, nilai)** yang memanggil setter Proxy, memicu subscriber. Aturan: target harus writable. **tetap** tidak bisa di-**simpan** (E3003), **turunan** juga tidak bisa

(E4004).

5.2 tambahkan — Polimorfik

`tambahkan` berperilaku berbeda tergantung tipe target.

TIPE TARGET	PERILAKU	CONTOH
Angka	Penjumlahan numerik	<code>tambahkan 5 ke counter</code> (counter += 5)
Array	Append item ke akhir	<code>tambahkan "item" ke daftar</code>
String	Concatenation (jika eksplisit)	<code>tambahkan "!" ke pesan</code>

```
KARSA · 12-TAMBAHKAN.KS

--! Angka: penjumlahan
data counter = 0
tambahkan 1 ke counter -- counter = 1
tambahkan 10 ke counter -- counter = 11

--! Array: append
data daftar = ["apel"]
tambahkan "mangga" ke daftar -- ["apel", "mangga"]

--! String: concatenation (warning jika ambigu)
data pesan = "Halo"
tambahkan " Dunia" ke pesan -- "Halo Dunia"
```

5.3 kurangi — Pengurangan Numerik

```
KARSA · 13-KURANGI.KS

--! Pengurangan dengan nilai eksplisit
data stok = 100
kurangi stok dengan 5 -- stok = 95

--! Default: kurangi 1
kurangi stok -- stok = 94 (kurangi 1)
```

● KURANGI HANYA UNTUK ANGKA

Berbeda dari `tambahkan` yang polimorfik, `kurangi` hanya berlaku untuk angka. Array dan string tidak punya operasi "kurangi" yang intuitif. Untuk hapus item array, gunakan `langsung:` dengan `splice` lalu trigger reaktivitas dengan spread (lihat Bab 6).

5.4 sisipkan — Array Insert

`sisipkan` adalah mutator khusus array yang selalu melakukan insert. Berbeda dari `tambahkan` yang juga bisa append, `sisipkan` eksplisit menyatakan intent "tambah ke koleksi".

```
data tugas = []

--! Sisipkan item ke array
sisipkan "Belajar KARSA" ke tugas
sisipkan "Makan siang" ke tugas

--! Setelah 2 sisipkan: tugas = ["Belajar KARSA", "Makan siang"]
```

5.5 Runtime Internals — `__setState`

```
// simpan 5 ke counter
__setState(counter, 5); // = counter.value = 5

// tambahkan 1 ke counter
__setState(counter, counter.value + 1);

// tambahkan "item" ke daftar (array)
daftar.value.push("item"); // trigger Proxy
```

◆ CATATAN PENGAJAR — BAB 5

DURASI 3 JAM • TEORI + PRAKTIK

TUJUAN BELAJAR

Peserta menguasai 4 mutator dan bisa memilih yang tepat berdasarkan tipe target dan intent.

SARAN MENGAJAR

Demo: counter dengan tambahkan/kurangi. Lalu demo array dengan sisipkan. Tunjukkan reaktivitas otomatis di UI.

PERTANYAAN DISKUSI

1. Kapan pakai tambahkan vs sisipkan? 2. Bisakah kurangi string? 3. Apa beda simpan vs tambahkan untuk angka?

KESALAHAN UMUM

- Pakai `kurangi` untuk array → error
- Bingung `tambahkan` vs `sisipkan` (overlapping)
- Lupa bahwa `simpan` ke `turunan` error (E4004)

TROUBLESHOOTING

- **E3003:** `simpan` ke `tetap`
- **E4004:** `simpan` ke `turunan`
- **E4101:** Target tidak writable

Reaktivitas Array & Object

Reaktivitas untuk nilai primitif (angka, string, boolean) sederhana — `simpan` langsung memicu subscriber. Tapi untuk array dan object, ada nuansa: Proxy tracking per-key, mutasi method (push/splice), dan pola spread trigger. Pahami perilaku ini agar reaktivitas selalu bekerja.

6.1 Deep Reactive — Tracking Per-Key

Setiap properti dari object reaktif juga di-track secara terpisah. Saat Anda mutasi `pengguna.skor`, hanya subscriber yang membaca `pengguna.skor` yang dipicu, bukan semua subscriber `pengguna`. Ini efisien.

```
KARSA · 15-OBJECT-TRACKING.KS

data pengguna = {
  nama: "Budi",
  skor: 0,
  level: 1
}

--! Hanya subscriber `pengguna.skor` yang dipicu
simpan 100 ke pengguna.skor

--! turunan yang baca `pengguna.nama` TIDAK re-evaluate
turunan nama_pengguna = pengguna.nama

--! turunan yang baca `pengguna.skor` re-evaluate
turunan skor_pengguna = pengguna.skor
```

6.2 Mutasi Array — sisipkan & tambahkan

Untuk append item ke array, gunakan `sisipkan` atau `tambahkan`. Keduanya memanggil `Array.push()` di balik layar, yang dipicu oleh Proxy setter pada properti `length` dan index baru.

```
KARSA · 16-ARRAY-TAMBAH.KS

data daftar = []

sisipkan "item 1" ke daftar -- ["item 1"]
sisipkan "item 2" ke daftar -- ["item 1", "item 2"]

--! turunan yang baca `daftar.panjang` re-evaluate
turunan jumlah = daftar.panjang
```

6.3 Hapus Item — Pola Spread Trigger

KARSA tidak punya mutator "hapus dari array" native. Untuk hapus item, gunakan `langsung:` dengan `splice`, lalu trigger reaktivitas dengan assign array baru via spread.

```
KARSA · 17-ARRAY-HAPUS.KS (DARI TODO-APP.HTML)

data semuaTugas = ["A", "B", "C"]

--! Hapus item index 1 ("B") via splice + spread
langsung:
  semuaTugas.value.splice(1, 1); -- hapus "B" dari array
  --! TRIGGER reaktivitas dengan assign array baru
  semuaTugas.value = [...semaTugas.value]; -- ["A", "C"]
```

● MENGAPA PERLU SPREAD?

`splice` memang menghapus item, tapi Proxy setter hanya terpicu ketika `.value` di-assign, bukan saat method array dipanggil. Untuk memastikan subscriber tahu array berubah, assign ulang dengan `[...array.value]`. Ini menciptakan reference baru, memicu Proxy setter, dan subscriber dijalankan.

6.4 Mutasi Nested Object

Untuk object nested, mutasi langsung pada property dalam tetap reaktif karena Proxy melewati setiap level.

```
KARSA · 18-NESTED-MUTASI.KS

data pengguna = {
  nama: "Budi",
  alamat: {
    kota: "Jakarta",
    kode_pos: "10110"
  }
}

--! Mutasi nested - reaktif
simpan "Bandung" ke pengguna.alamat.kota
simpan "40123" ke pengguna.alamat.kode_pos

--! turunan yang baca nested akan re-evaluate
turunan lokasi = pengguna.alamat.kota + " " + pengguna.alamat.kode_pos
```

6.5 Limitation & Workaround

KASUS	LIMITATION	WORKAROUND
Hapus item array	Tidak ada mutator native	<code>langsung:</code> + splice + spread
Sort/filter array	Method tidak otomatis trigger	Assign hasil ke <code>.value</code>
Map/reduce	Method tidak otomatis trigger	Assign hasil ke variabel lain
Set/Map	Tidak ada Proxy tracking	Pakai array atau object biasa

◆ CATATAN PENGAJAR — BAB 6

DURASI 3 JAM • TEORI + PRAKTIK

TUJUAN BELAJAR

Peserta memahami reaktivitas array & object, pola spread trigger, dan limitation KARSA.

SARAN MENGAJAR

Demo todo-app dengan hapus. Tunjukkan tanpa spread, UI tidak update. Lalu tambahkan spread, UI update. Visualisasikan reference change.

KESALAHAN UMUM

- Lupa spread setelah splice → UI tidak update
- Pakai Set/Map → tidak reaktif
- Bingung kenapa `filter` tidak trigger

TROUBLESHOOTING

- UI tidak update setelah splice → tambahkan spread
- Nested mutation tidak trigger → pastikan `data` (bukan `tetap`)

PERTANYAAN DISKUSI

1. Mengapa `splice` tidak otomatis trigger? 2. Apakah Set/Map bisa dibuat reaktif? 3. Bagaimana cara sort array reaktif?

BAB 7

Dependency Graph — Visualisasi

KARSA CLI menyediakan `karsa graph` untuk memvisualisasikan dependency graph dari `turunan` dan `saat` watcher. Tool ini penting untuk debugging reaktivitas — Anda bisa lihat state mana yang bergantung pada apa, dan mendeteksi cycle yang berbahaya.

7.1 Perintah `karsa graph`

```
# Output teks/JSON
karsa graph app.ks
karsa graph app.ks --json > graph.json
```

BASH • CLI

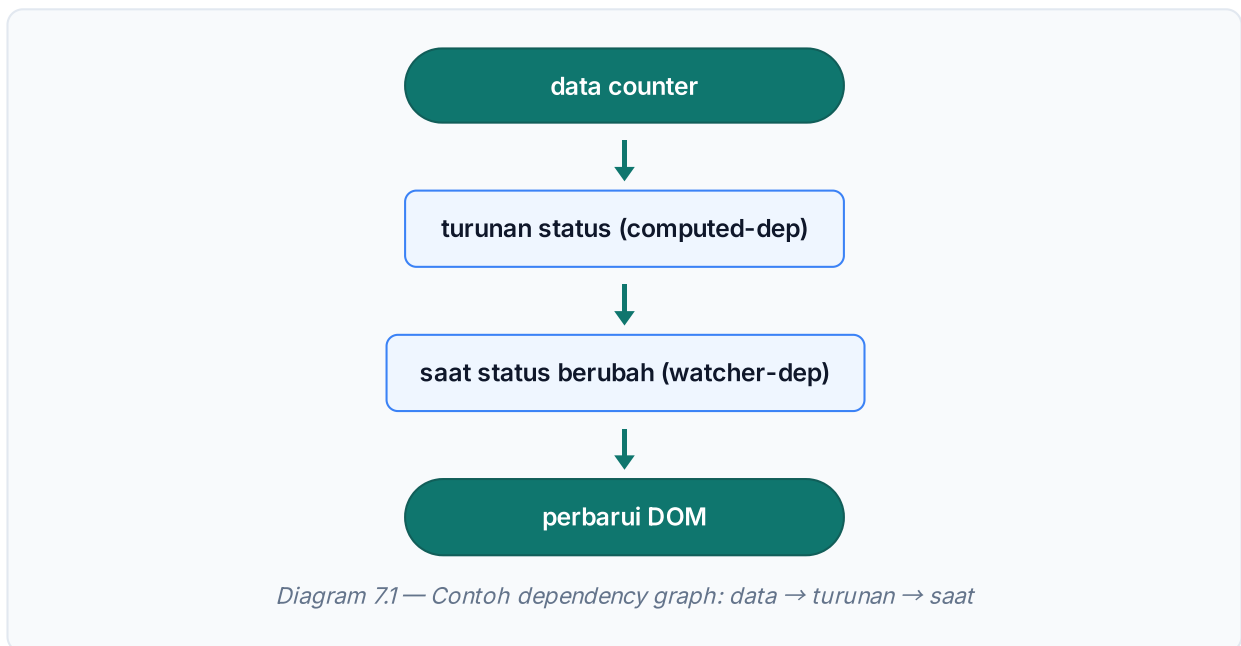
7.2 Output JSON — Structure

```
JSON · OUTPUT KARSA GRAPH --JSON

{
  "command": "graph",
  "version": "0.3.1",
  "symbols": [
    { "id": "sym_1", "name": "counter", "kind": "data", "isReactive": true },
    { "id": "sym_2", "name": "status", "kind": "turunan", "isComputed": true }
  ],
  "dependencies": [
    { "from": "status", "to": "counter", "kind": "computed-dep" }
  ],
  "cycles": []
}
```

7.3 Membaca Graph

Setiap dependency punya `from` (yang membaca) dan `to` (yang dibaca). `kind` bisa `computed-dep` (turunan membaca data) atau `watcher-dep` (saat watcher memantau data). Graph membentuk DAG (Directed Acyclic Graph) jika tidak ada cycle.



7.4 Anti-Pattern — Circular Dependency (E4201)

Cycle terjadi ketika A bergantung pada B, dan B bergantung pada A. KARSA mendeteksi cycle via DFS (Depth-First Search) di `dependency-graph.js` dan mengeluarkan error E4201.


```

--! ANTI-PATTERN: circular dependency
turunan a = b + 1 -- a butuh b
turunan b = a - 1 -- b butuh a → CYCLE!

--! Error E4201: Dependency cycle pada data turunan
--! Saran: Ubah salah satu ekspresi turunan agar
--!         tidak saling bergantung secara melingkar.

```

7.5 Deteksi Cycle via DFS

KARSA menggunakan algoritma DFS dengan tracking `active` set untuk mendeteksi cycle. Jika node yang sedang dikunjungi ditemukan lagi di active set, cycle terdeteksi.

```

function detectCycles(graph) {
  const visited = new Set();
  const active = new Set();
  const cycles = [];

  function dfs(node) {
    visited.add(node);
    active.add(node);

    const nexts = graph.get(node) || [];
    for (const next of nexts) {
      if (!visited.has(next)) {
        dfs(next);
      } else if (active.has(next)) {
        // CYCLE terdeteksi!
        cycles.push({ symbolIds: [...] });
      }
    }
    active.delete(node);
  }

  // ... mulai DFS dari setiap node
}

```

TUJUAN BELAJAR

Peserta bisa membaca dependency graph, mendeteksi cycle, dan menggunakan karsa graph untuk debugging.

SARAN MENGAJAR

Demo: kode dengan 5 data + 3 turunan.
Jalankan karsa graph --json, lihat output.
Lalu buat cycle, lihat error E4201.

KESALAHAN UMUM

- Membuat turunan saling bergantung → E4201
- Bingung arah dependency (from vs to)
- Lupa bahwa watcher juga masuk graph

TROUBLESHOOTING

- **E4201**: pecah cycle dengan refactor salah satu turunan jadi data
- Graph kosong → cek apakah ada turunan/saat

PERTANYAAN DISKUSI

1. Apa beda cycle di turunan vs cycle di saat watcher? 2. Bagaimana cara visualkan graph selain JSON? 3. Bisakah graph dipakai untuk optimasi?

BAB 8

Inspect Mode — Debug Reaktivitas

`karsa inspect` menampilkan SemanticSymbol — metadata lengkap tentang setiap simbol (data, turunan, fungsi, komponen) di file KARSA. Dengan inspect, Anda bisa lihat readCount, writeCount, isReactive, isComputed — informasi penting untuk debugging reaktivitas.

8.1 Perintah karsa inspect

```
# Output teks/JSON
karsa inspect app.ks
karsa inspect app.ks --json > inspect.json
```

8.2 Output — SemanticSymbol Fields

FIELD	TIPE	KETERANGAN
<code>id</code>	string	ID unik simbol (mis. <code>sym_1</code>)
<code>name</code>	string	Nama variabel/komponen/fungsi
<code>kind</code>	string	Jenis: <code>data</code> , <code>turunan</code> , <code>tetap</code> , <code>ubah</code> , <code>fungsi</code> , <code>komponen</code> , <code>parameter</code>
<code>isReactive</code>	boolean	True jika simbol reaktif (data/turunan)
<code>isWritable</code>	boolean	True jika bisa di- <code>simpan</code> (data, bukan turunan)
<code>isComputed</code>	boolean	True jika turunan
<code>isParameter</code>	boolean	True jika parameter komponen/fungsi
<code>isComponent</code>	boolean	True jika komponen
<code>isFunction</code>	boolean	True jika fungsi
<code>readCount</code>	number	Jumlah kali simbol dibaca
<code>writeCount</code>	number	Jumlah kali simbol ditulis
<code>shadowedSymbolId</code>	string?	ID simbol yang di-shadow (jika ada)

8.3 Contoh Output

```
JSON · OUTPUT KARSA INSPECT --JSON

{
  "command": "inspect",
  "symbols": [
    {
      "id": "sym_1",
      "name": "counter",
      "kind": "data",
      "isReactive": true,
      "isWritable": true,
      "isComputed": false,
      "readCount": 3,
      "writeCount": 2
    },
    {
      "id": "sym_2",
      "name": "status_counter",
      "kind": "turunan",
      "isReactive": true,
      "isWritable": false,
      "isComputed": true,
      "readCount": 1,
      "writeCount": 0
    }
  ]
}
```

8.4 Use Case Debugging

GEJALA	CEK DI INSPECT	DIAGNOSIS
State tidak update UI	<code>isReactive</code>	Jika false, ganti <code>ubah</code> ke <code>data</code>
State tidak terbaca	<code>readCount = 0</code>	W4101: simbol dideklarasikan tapi tidak dipakai
Write tapi UI tidak update	<code>writeCount > 0,</code> <code>readCount = 0</code>	W4102: tulis tapi tidak dibaca
Turunan tidak re-evaluate	<code>isComputed,</code> <code>readCount</code>	Pastikan ada yang baca turunan
Watcher tidak fire	<code>isReactive</code> <code>target</code>	W4104: target bukan data reaktif

TUJUAN BELAJAR

Peserta bisa memakai karsa `inspect` untuk debugging, membaca `SemanticSymbol`, dan mendiagnosis masalah reaktivitas.

SARAN MENGAJAR

Demo: kode dengan bug reaktivitas (state tidak update). Jalankan `inspect`, tunjukkan `readCount=0`. Fix dengan ganti ubah ke `data`, lihat `readCount` naik.

KESALAHAN UMUM

- Tidak tahu `readCount/writeCount` bisa deteksi `unused state`
- Bingung beda `isReactive` vs `isWritable`
- Lupa cek `inspect` saat debugging reaktivitas

TROUBLESHOOTING

- **W4101:** Hapus simbol yang `readCount=0`
- **W4102:** Pastikan `write` dibaca
- **W4103:** Hapus mutasi yang tidak pernah dibaca

PERTANYAAN DISKUSI

1. Apa beda `isReactive` vs `isWritable`? 2. Kapan `readCount=0` wajar? 3. Bagaimana `inspect` membantu optimasi?

BAB 9

Mini-Proyek: Counter Multi-State dengan Computed

Saatnya mengintegrasikan semua konsep Modul 4 dalam satu aplikasi nyata. Kita akan membangun **Counter Multi-State** — diadaptasi dari `example/index.ks` di repo `dazep01`. Aplikasi ini menggabungkan `data`, `turunan`, `saat`, dan empat mutator dalam satu file lengkap dengan UI reaktif.

9.1 Spesifikasi

- **Counter:** Angka yang bisa tambah/reset.
- **Status:** "Genap" atau "Ganjil" — otomatis dari counter (turunan).
- **Pesan selamat:** "Halo, <nama>!" — otomatis dari `nama_pengguna` (turunan).
- **UI reaktif:** Semua perubahan state langsung tercermin di DOM tanpa manual update.

9.2 Analisis Alur

KEBUTUHAN	SINTAKS KARSA	ALASAN
State counter	<code>data counter = 0</code>	Reaktif, perlu di-mutasi
State nama	<code>data nama_pengguna = "Kawan"</code>	Reaktif, bisa diubah
Status genap/ganjil	<code>turunan status_counter = jalankan cekGenap dengan counter</code>	Auto-compute dari counter
Pesan selamat	<code>turunan pesan_selamat = "Halo, " + nama_pengguna + "!"</code>	Auto-compute dari nama
Sinkron UI counter	<code>saat counter berubah: perbarui teks ...</code>	Side-effect: update DOM
Sinkron UI pesan	<code>saat pesan_selamat berubah: perbarui teks ...</code>	Side-effect: update DOM

9.3 Tahap 1 — State & UI Dasar

```
KARSA · 20-COUNTER-TAHAP-1.KS

--! State dasar
data counter = 0
data nama_pengguna = "Kawan"

--! UI statis
buat div.app-container
  buat h1#judul → teks: "Halo, Kawan!"

  buat div.counter-box
    buat p#nilai-text.text-bold → teks: "Nilai saat ini: 0"
    buat p#status-text → teks: "Status angka: Genap"

    buat tombol#btn-tambah → teks: "Tambah Angka"
      ketika diklik → tambahkan 1 ke counter

    buat tombol#btn-reset → teks: "Reset"
      ketika diklik → simpan 0 ke counter
```

9.4 Tahap 2 — Tambah turunan

```
KARSA · 21-COUNTER-TAHAP-2.KS

--! Fungsi JS untuk cek ganjil/genap
langsung:
  function cekGenap(n) {
    return n % 2 === 0 ? "Genap" : "Ganjil";
  }

data counter = 0
data nama_pengguna = "Kawan"

--! turunan: status berubah otomatis saat counter berubah
turunan status_counter = jalankan cekGenap dengan counter

--! turunan: pesan berubah saat nama_pengguna berubah
turunan pesan_selamat = "Halo, " + nama_pengguna + "!"
```

9.5 Tahap 3 — Tambah saat Watcher

```
KARSA · 22-COUNTER-TAHAP-3.KS

--! Sinkron counter ke UI
saat counter berubah:
  perbarui teks "#nilai-text" → "Nilai saat ini: " + counter
  perbarui teks "#status-text" → "Status angka: " + status_counter

--! Sinkron pesan ke UI
saat pesan_selamat berubah:
  perbarui teks "#judul" → pesan_selamat
```

9.6 Versi Final Lengkap

```
KARSA · 23-COUNTER-FINAL.KS (VERSI LENGKAP DARI EXAMPLE/INDEX.KS)

--! Helper JS untuk cek genap/ganjil
langsung:
  function cekGenap(n) {
    return n % 2 === 0 ? "Genap" : "Ganjil";
  }

--! State reaktif
data counter = 0
data nama_pengguna = "Kawan"

--! Computed properties
turunan status_counter = jalankan cekGenap dengan counter
turunan pesan_selamat = "Halo, " + nama_pengguna + "!"

--! UI
buat div.app-container
  buat h1#judul → teks: pesan_selamat

  buat div.counter-box
    buat p#nilai-text.text-bold → teks: "Nilai saat ini: " + counter
    buat p#status-text → teks: "Status angka: " + status_counter

    buat tombol#btn-tambah → teks: "Tambah Angka"
      ketika diklik → tambahkan 1 ke counter

    buat tombol#btn-reset → teks: "Reset"
      ketika diklik → simpan 0 ke counter

--! Watcher: sinkron counter ke UI
saat counter berubah:
  perbarui teks "#nilai-text" → "Nilai saat ini: " + counter
  perbarui teks "#status-text" → "Status angka: " + status_counter

--! Watcher: sinkron pesan ke UI
saat pesan_selamat berubah:
  perbarui teks "#judul" → pesan_selamat
```

9.7 Diskusi Penyempurnaan

- **Persistence:** Simpan counter ke `localStorage` via `jalankan`.
- **Multiple counters:** Pakai array of objects, iterasi dengan `ulangi`.
- **Derived UI:** Sembunyikan tombol reset jika counter=0 (turunan + saat).
- **Animation:** Tambah transisi CSS saat counter berubah (saat + `perbarui kelas`).
- **History:** Simpan riwayat perubahan counter ke array, tampilkan di UI.

TUJUAN BELAJAR

Peserta mampu mengintegrasikan data, turunan, saat, dan mutator dalam satu aplikasi reaktif lengkap.

SARAN MENGAJAR

Minta siswa selesaikan Tahap 1 dalam 30 menit. Tahap 2 + 3 sebagai PR dengan deadline 1 minggu. Review code bersama di pertemuan berikutnya.

KESALAHAN UMUM

- Lupa `saat` watcher → UI tidak update
- Coba `simpan` ke turunan → E4004
- Bingung kapan pakai turunan vs saat

TROUBLESHOOTING

- UI tidak update → cek `saat` watcher ada
- Turunan tidak re-evaluate → cek dependency terbaca
- `karsa inspect` untuk lihat `readCount/writeCount`

PERTANYAAN DISKUSI

1. Apa beda jika pakai `ubah` vs `data` untuk counter? 2. Bagaimana cara persist counter ke `localStorage`? 3. Bisakah turunan memanggil fungsi JS?

LATIHAN

Latihan Soal — 15 Soal Bertingkat

15 soal bertingkat untuk menguji pemahaman reaktivitas KARSA. Soal dibagi tiga level: **mudah** (sintaks dasar data/turunan/saat), **sedang** (kombinasi computed + watcher + mutator), dan **sulit** (debugging cycle, optimasi re-render, anti-pattern).

A Level Mudah

L1 • MUDAH**MUDAH**

Deklarasikan `data counter = 0` lalu buat `turunan` yang bernilai `"Genap"` jika counter genap, `"Ganjil"` jika tidak.

L2 • MUDAH**MUDAH**

Tulis pernyataan `saat` yang menampilkan `notifikasi` "Counter berubah!" setiap kali `counter` berubah.

L3 • MUDAH**MUDAH**

Deklarasikan `data daftar = []` lalu tambahkan 3 item ("A", "B", "C") menggunakan `sisipkan`.

L4 • MUDAH**MUDAH**

Deklarasikan `data harga = 10000` dan `data jumlah = 3`, lalu buat `turunan` `total = harga * jumlah`.

L5 • MUDAH

MUDAH

Buat `saat` watcher yang mengupdate teks elemen `#angka` menjadi nilai `counter` saat `counter` berubah.

B Level Sedang

L6 • SEDANG

SEDANG

Buat aplikasi counter dengan `data`, `turunan` (status genap/ganjil), dan `saat` watcher yang update UI. Tombol "Tambah" memakai `tambahkan 1 ke counter`.

L7 • SEDANG

SEDANG

Diberikan `data pengguna = { nama: "Budi", skor: 0 }`. Buat `turunan` yang menampilkan "Skor Budi: 0". Mutasi `pengguna.skor` dengan `simpan 100 ke pengguna.skor` — apakah turunan re-evaluate? Jelaskan.

L8 • SEDANG

SEDANG

Buat todo list sederhana: `data tugas = []`, tambah tugas via `sisipkan`, hapus tugas via `langsung: + splice + spread`. Pasang `saat` watcher untuk re-render list.

L9 • SEDANG

SEDANG

Buat `turunan` yang memanggil fungsi `format_rupiah(n)` (native) untuk memformat `data harga = 15000`. Pastikan tidak melanggar aturan no-side-effect.

L10 • SEDANG

SEDANG

Buat `saat` watcher yang membatasi `counter` maksimal 10. Jika `counter > 10`, simpan 10 ke counter. Hindari infinite loop.

C Level Sulit

L11 • SULIT

SULIT

Identifikasi bug: `data counter = 0`, `ubah backup = 0`, `saat counter berubah: simpan counter ke backup`. Mengapa `backup` tidak bisa dipakai untuk UI reaktif? Fix dengan ganti `ubah` ke `data`.

L12 • SULIT

SULIT

Debug cycle: `turunan a = b + 1`, `turunan b = a - 1`. Apa error yang muncul? Bagaimana cara fix?

L13 • SULIT

SULIT

Optimasi: aplikasi punya 1000 item di array, setiap kali array diubah, **saat** watcher re-render semua item. Bagaimana cara mengurangi overhead? Petunjuk: DocumentFragment, virtual scrolling, atau pagination.

L14 • SULIT

SULIT

Refaktor kode berikut yang punya watcher infinite loop: **saat counter berubah: tambahkan 1 ke counter**. Berikan solusi dengan kondisi exit dan jelaskan kenapa loop terjadi.

L15 • SULIT

SULIT

Buat aplikasi shopping cart: **data keranjang = []**, **turunan total_harga** (sum dari harga semua item), **saat keranjang berubah** untuk update UI. Tambah fungsi tambah/hapus item dengan reaktivitas yang benar.

SOLUSI

Solusi Latihan Lengkap

Berikut solusi untuk semua 15 soal. Format: kode solusi, penjelasan singkat, alternatif, dan pembahasan error umum.

SOLUSI L1

MUDAH

```
data counter = 0

turunan status = jika counter % 2 == 0: "Genap" jika tidak: "Ganjil"
```

Penjelasan: turunan re-evaluate otomatis saat counter berubah. % adalah modulo. **Alternatif:** pakai jalankan ke fungsi JS cekGenap. **Error umum:** lupa jika tidak di akhir rantai.

SOLUSI L2

MUDAH

```
data counter = 0

saat counter berubah:
  tampilkan notifikasi "Counter berubah!"
```

Penjelasan: saat watcher fire setiap kali counter berubah (batched). notifikasi adalah toast yang auto-dismiss. **Error umum:** coba pakai ubah sebagai target → W4104.

SOLUSI L3

MUDAH

```
data daftar = []

sisipkan "A" ke daftar
sisipkan "B" ke daftar
sisipkan "C" ke daftar
-- daftar = ["A", "B", "C"]
```

Penjelasan: sisipkan selalu append ke array. Reaktivitas otomatis. **Alternatif:** tambahkan juga bisa untuk array. **Error umum:** lupa ke setelah item.

SOLUSI L4

MUDAH

```
data harga = 10000
data jumlah = 3

turunan total = harga * jumlah
```

Penjelasan: turunan bergantung pada harga dan jumlah otomatis. Saat salah satu berubah, total re-evaluate. **Error umum:** coba simpan ke total → E4004.

SOLUSI L5

MUDAH

```
data counter = 0

saat counter berubah:
  perbarui teks "#angka" → counter
```

Penjelasan: Side-effect (update DOM) wajib pakai saat, bukan turunan. **Alternatif:** pakai binding otomatis jika ada. **Error umum:** pakai turunan untuk side-effect → E4002.

SOLUSI L6

SEDANG

```
data counter = 0

turunan status = jika counter % 2 == 0: "Genap" jika tidak: "Ganjil"

buat tombol#btn → teks: "Tambah"
  ketika diklik → tambahkan 1 ke counter

buat p#angka
buat p#status

saat counter berubah:
  perbarui teks "#angka" → "Counter: " + counter
  perbarui teks "#status" → "Status: " + status
```

Penjelasan: Gabungan data + turunan + saat + mutator. tambahkan mutate counter, turunan status re-evaluate otomatis, saat update UI. **Error umum:** lupa saat → UI tidak update.

```
data pengguna = { nama: "Budi", skor: 0 }

turunan info = "Skor " + pengguna.nama + ": " + pengguna.skor

--! Mutasi nested
simpan 100 ke pengguna.skor

--! Ya, turunan re-evaluate karena deep reactive.
--! Proxy track `pengguna.skor` sebagai dependency terpisah.
```

Penjelasan: KARSA deep reactive. Mutasi `pengguna.skor` memicu subscriber yang membaca `pengguna.skor`. `turunan info` re-evaluate. **Error umum:** mengira hanya `simpan ... ke pengguna` (whole object) yang trigger.

```
data tugas = []

buat masukan#input-tugas
buat tombol#btn-tambah → teks: "Tambah"
  ketika diklik:
    ambil nilai dari #input-tugas → simpan ke teks
    sisipkan teks ke tugas

buat ul#list

saat tugas berubah:
  kosongkan #list
  ulangi t dari tugas:
    buat li
      buat span → teks: t
      buat tombol → teks: "Hapus"
        ketika diklik:
          langsung:
            const idx = tugas.value.indexOf(t);
            tugas.value.splice(idx, 1);
            tugas.value = [...tugas.value]; --! TRIGGER
```

Penjelasan: sisipkan untuk tambah, `splice` + `spread` untuk hapus (trigger reaktivitas). saat tugas berubah re-render list. **Error umum:** lupa `spread` setelah `splice` → UI tidak update.

SOLUSI L9

SEDANG

```
KARSA · SOLUSI-L9.KS

fungsi format_rupiah(n: angka) → teks:
  kembalikan "Rp" + n.toLocaleString("id-ID")

data harga = 15000

--! turunan memanggil fungsi pure - aman
turunan harga_format = format_rupiah(harga)
```

Penjelasan: format_rupiah pure (hanya format angka, no side-effect). Aman dipanggil di turunan.

Alternatif: pakai jalankan ke Intl.NumberFormat JS. **Error umum:** fungsi dengan console.log → E4002.

SOLUSI L10

SEDANG

```
KARSA · SOLUSI-L10.KS

data counter = 0

saat counter berubah:
  jika counter > 10:
    simpan 10 ke counter --! cap di 10
```

Penjelasan: Kondisi jika counter > 10 mencegah infinite loop. Setelah simpan 10 ke counter, nilai sudah 10, kondisi tidak terpenuhi lagi. **Error umum:** tanpa kondisi → loop forever.

SOLUSI L11

SULIT

```
KARSA · SOLUSI-L11.KS

--! BUG: `ubah` tidak reaktif
data counter = 0
ubah backup = 0

saat counter berubah:
  simpan counter ke backup --! backup berubah, tapi UI tidak update

--! FIX: ganti `ubah` ke `data`
data counter = 0
data backup = 0 --! sekarang reaktif

saat counter berubah:
  simpan counter ke backup

--! UI yang baca `backup` akan update otomatis
```

Penjelasan: ubah tidak di-wrap Proxy, jadi mutasi tidak trigger subscriber. Untuk state UI, wajib data. **Diagnosis:** jalankan `karsa inspect —isReactive: false` untuk backup.

SOLUSI L12

SULIT

```
KARSA · SOLUSI-L12.KS

--! ERROR: E4201 - Dependency cycle
turunan a = b + 1
turunan b = a - 1

--! FIX: pecah cycle dengan buat `b` jadi data
data b = 0
turunan a = b + 1
```

Penjelasan: a butuh b, b butuh a → cycle. KARSA deteksi via DFS. Fix: salah satu jadikan data (sumber kebenaran), turunan hanya turunan satu arah. **Alternatif:** pakai data untuk keduanya lalu sinkron via saat.


```
KARSA · SOLUSI-L13.KS

--! OPTIMASI 1: DocumentFragment (1 reflow, bukan 1000)
saat items berubah:
  langsung:
    const frag = document.createDocumentFragment();
    items.value.forEach((item) => {
      const li = document.createElement("li");
      li.innerText = item;
      frag.appendChild(li);
    });
    document.querySelector("#list").replaceChildren(frag);

--! OPTIMASI 2: Pagination (hanya render 50 item)
data halaman = 1
turunan items_terlihat = items.slice((halaman - 1) * 50, halaman * 50)
```

Penjelasan: (1) DocumentFragment batch 1000 appendChild jadi 1 reflow. (2) Pagination hanya render 50 item per halaman. (3) Virtual scrolling (advanced) — render hanya item yang terlihat di viewport. **Trade-off:** kompleksitas kode naik.

```
KARSA · SOLUSI-L14.KS

--! ANTI-PATTERN: infinite loop
data counter = 0

saat counter berubah:
  tambahkan 1 ke counter --! loop forever

--! FIX: kondisi exit
saat counter berubah:
  jika counter < 100:
    tambahkan 1 ke counter
  --! Setelah counter=100, kondisi false, loop berhenti
```

Penjelasan: Mutasi state yang sama di watcher menyebabkan watcher fire lagi, mutate lagi, fire lagi — infinite. Kondisi exit menghentikan rantai. **Alternatif:** pakai selama loop eksplisit untuk iterasi sampai target.

```
data keranjang = []

--! turunan: total harga (sum dari item.harga * item.jumlah)
turunan total_harga = jalankan hitungTotal dengan keranjang

langsung:
  function hitungTotal(arr) {
    return arr.reduce((sum, item) => sum + item.harga * item.jumlah, 0);
  }

--! UI
buat div#cart
  buat h2 → teks: "Keranjang"
  buat ul#items
  buat p#total → teks: "Total: Rp" + total_harga
  buat tombol#btn-add → teks: "Tambah Item"
    ketika diklik:
      sisipkan { nama: "Item", harga: 10000, jumlah: 1 } ke keranjang

--! Watcher: re-render list saat keranjang berubah
saat keranjang berubah:
  kosongkan #items
  ulangi item dari keranjang:
    buat li
      buat span → teks: item.nama + " x" + item.jumlah + " = Rp" + (item.harga
      buat tombol → teks: "Hapus"
        ketika diklik:
          langsung:
            const idx = keranjang.value.indexOf(item);
            keranjang.value.splice(idx, 1);
            keranjang.value = [...keranjang.value];

--! Watcher: update total
saat total_harga berubah:
  perbarui teks "#total" → "Total: Rp" + total_harga
```

Penjelasan: Gabungan data (keranjang), turunan (total_harga via jalankan ke reduce), saat watcher untuk re-render. **Key points:** (1) turunan pakai fungsi pure reduce via jalankan; (2) hapus pakai splice + spread; (3) dua watcher terpisah untuk list dan total.

Referensi Cepat

Cheat sheet satu halaman berisi semua kata kunci Modul 4.

Deklarasi State

```
--! reaktif
data x = 0

--! konstanta
tetap PI = 3.14

--! mutable non-reaktif
ubah temp = 0

--! type hint
data umur: angka = 25
```

Mutator

```
--! assignment
simpan <nilai> ke <var>

--! tambah (angka/array/string)
tambahkan <n> ke <var>

--! kurangi (angka saja)
kurangi <var> dengan <n>

--! sisipkan (array)
sisipkan <item> ke <array>
```

Computed & Watcher

```
--! computed (read-only)
turunan y = x * 2

--! watcher (side-effect)
saat x berubah:
  perbarui teks "#e1" → x
```

Tooling CLI

```
--! visualisasi dependency
karsa graph file.ks
karsa graph file.ks --json

--! debug SemanticSymbol
karsa inspect file.ks
karsa inspect file.ks --json

--! lint + error
karsa check file.ks
```

Perbandingan Framework Reaktif

Tabel padanan KARSA dengan framework reaktif populer: Vue, React, dan Solid.

KONSEP	KARSA	VUE 3	REACT	SOLID
State reaktif	<code>data x = 0</code>	<code>ref(0)</code>	<code>useState(0)</code>	<code>createSignal(0)</code>
Akses nilai	<code>x</code>	<code>x.value</code>	<code>x</code>	<code>x()</code>
Set nilai	<code>simpan 5 ke x</code>	<code>x.value = 5</code>	<code>setX(5)</code>	<code>setX(5)</code>
Computed	<code>turunan y = x*2</code>	<code>computed(() => ...)</code>	<code>useMemo(..., [x])</code>	<code>createMemo(() => ...)</code>
Watcher	<code>saat x berubah:</code>	<code>watch(x, ...)</code>	<code>useEffect(..., [x])</code>	<code>createEffect(() => ...)</code>
Deep reactive	Default (Proxy)	<code>reactive(obj)</code>	Manual spread	Default (Proxy)
Dep tracking	Otomatis	Otomatis	Manual array	Otomatis
Batching	Microtask	nextTick	Concurrent	Microtask
Auto-unwrap	Ya	Tidak (perlu .value)	Ya	Tidak (perlu call)
Boilerplate	Minimal	Sedang	Banyak	Sedang
Virtual DOM	Tidak	Ya	Ya	Tidak
Compiled output	Vanilla DOM API	Render functions	JSX	Vanilla DOM API

LAMPIRAN C

Schema SemanticSymbol & Dependency

Field lengkap dari output `karsa inspect` (SemanticSymbol) dan `karsa graph` (Dependency).

SemanticSymbol

FIELD	TIPE	KETERANGAN
<code>id</code>	string	ID unik simbol (mis. <code>sym_1</code>)
<code>name</code>	string	Nama variabel/komponen/fungsi
<code>kind</code>	enum	<code>data</code> <code>turunan</code> <code>tetap</code> <code>ubah</code> <code>fungsi</code> <code>komponen</code> <code>parameter</code>
<code>loc</code>	SourceLocation	Lokasi baris:kolom deklarasi
<code>scope</code>	string?	Nama scope (global/komponen/fungsi)
<code>scopeId</code>	string?	ID scope
<code>isReactive</code>	boolean	True jika simbol reaktif
<code>isWritable</code>	boolean	True jika bisa di- <code>simpan</code>
<code>isComputed</code>	boolean	True jika turunan
<code>isParameter</code>	boolean	True jika parameter fungsi/komponen
<code>isComponent</code>	boolean	True jika komponen
<code>isFunction</code>	boolean	True jika fungsi
<code>readCount</code>	number	Jumlah kali dibaca
<code>writeCount</code>	number	Jumlah kali ditulis
<code>shadowedSymbolId</code>	string?	ID simbol yang di-shadow

Dependency

FIELD	TIPE	KETERANGAN
<code>from</code>	string	Nama simbol yang membaca (mis. <code>total</code>)
<code>fromSymbolId</code>	string?	ID simbol from
<code>to</code>	string	Nama simbol yang dibaca (mis. <code>harga</code>)
<code>toSymbolId</code>	string?	ID simbol to
<code>kind</code>	enum	<code>computed-dep</code> <code>watcher-dep</code>
<code>loc</code>	SourceLocation	Lokasi akses dependency

Error Codes Reaktivitas

Daftar error codes yang relevan untuk Modul 4 (Reaktivitas).

KODE	SEVERITY	STAGE	PESAN	SARAN
E3003	ERROR	Resolver	Variabel tetap tidak dapat diubah	Gunakan <code>ubah</code> / <code>data</code> jika perlu mengubah
E4002	ERROR	Analyzer	Ekspresi turunan tidak boleh mengandung side-effect	Hapus aksi imperatif dari turunan
E4004	ERROR	Analyzer	Data turunan bersifat read-only	Gunakan <code>data</code> jika perlu writable
E4101	ERROR	Analyzer	Target tidak dapat ditulis (isWritable false)	Pastikan target adalah <code>data</code>
E4201	ERROR	Analyzer	Dependency cycle pada data turunan	Pecah cycle, jadikan salah satu <code>data</code>
W3003	WARNING	Resolver	Watcher target bukan data reaktif	Gunakan <code>data</code> sebagai target watcher
W4001	WARNING	Analyzer	Type hint tidak cocok dengan nilai	Sesuaikan type hint atau nilai
W4101	WARNING	Analyzer	Simbol dideklarasikan tapi tidak pernah digunakan	Hapus deklarasi atau gunakan simbol
W4102	WARNING	Analyzer	Simbol ditulis tapi tidak pernah dibaca	Pastikan nilai yang ditulis dibaca
W4103	WARNING	Analyzer	Data reaktif dimutasi tapi tidak pernah dibaca	Hapus mutasi atau ubah jadi <code>ubah</code>
W4104	WARNING	Analyzer	Watcher target bukan data reaktif (menurut analyzer)	Gunakan <code>data</code> / <code>turunan</code> sebagai target

Selamat!

Anda telah menyelesaikan Modul 4 KARSA

Anda kini menguasai reaktivitas KARSA — signal-based, computed properties, watcher, dependency graph, dan tooling profesional. Ini adalah fondasi yang akan dipakai di seluruh aplikasi reaktif yang Anda bangun.

YANG TELAH ANDA PELAJARI

- ✓ Bab 1 — Filosofi signal-based reactivity
- ✓ Bab 2 — **data** state reaktif + deep reactive
- ✓ Bab 3 — **turunan** computed + auto-deps
- ✓ Bab 4 — **saat** watcher + batching
- ✓ Bab 5 — 4 mutator (simpan/tambahkan/kurangi/sisipkan)
- ✓ Bab 6 — Reaktivitas array & object + spread
- ✓ Bab 7 — **karsa graph** + deteksi cycle
- ✓ Bab 8 — **karsa inspect** + SemanticSymbol
- ✓ Bab 9 — Mini-proyek Counter Multi-State

"Reaktivitas bukan tentang membuat UI update otomatis, tapi tentang mendeklarasikan kebenaran — sisanya otomatis."